

Nama : Gilbran Mahdavia Raja
NRP : 5025241134

TUGAS 3 PBO C Analisis Perbandingan 2 Solusi

Kode 1 (Penyelesaian Saya)	Kode 2 (Penyelesaian Bapak Rully)
<pre>#include <stdio.h> #include <queue> #include <unordered_map> using namespace std; int main(){ int t; scanf("%d", &t); unordered_map<int, int> team; unordered_map<int, queue<int>> members; queue<int> torder; unordered_map<int, bool> inqueue; for (int i = 1; i <= t; i++){ int n; scanf("%d", &n); for (int j = 0; j < n; j++){ int student; scanf("%d", &student); team[student] = i; } } char cmd[10]; while (scanf("%s", cmd) != EOF){ if (cmd[0] == 'E'){ int ID; scanf("%d", &ID); int tid = team[ID]; if (!inqueue[tid]){ torder.push(tid); inqueue[tid] = true; } members[tid].push(ID); } else{ int tid = torder.front(); int student = members[tid].front(); members[tid].pop(); printf("%d\n", student); if (members[tid].empty()){ torder.pop(); inqueue[tid] = false; } } } return 0; }</pre>	<pre>#include <stdio> #include <queue> #include <map> #include <algorithm> using namespace std; queue<int> team, each_team[1001]; map<int, int> on_team; int main() { int t; scanf("%d", &t); for (int i = 0; i < t; i++) { int n; scanf("%d", &n); for (int j = 0; j < n; j++) { int m; scanf("%d", &m); on_team[m] = i; } } char cmd[10]; while (scanf("%s", cmd) != EOF) { if (cmd[0] == 'D') { int can_team = team.front(); printf("%d\n", each_team[can_team].front()); each_team[can_team].pop(); if (each_team[can_team].empty()) { team.pop(); } } else { int id; scanf("%d", &id); int id_team = on_team[id]; if (each_team[id_team].empty()) { team.push(id_team); } each_team[id_team].push(id); } } return 0; }</pre>

Nama : Gilbran Mahdavia Raja
NRP : 5025241134

Analisis:

Persamaan antara kode 1 dan kode 2 adalah:

1. Keduanya membuat mapping dari ID anggota ke tim (team[student] di kode 1, on_team[m] di kode 2).
2. Keduanya membuat queue per tim (members[tid] di kode 1, each_team[id_team] di kode 2).
3. Keduanya punya queue global berisi urutan tim (torder di kode 1, team di kode 2).
4. Mekanisme:
 - a. Enqueue (E): masukkan anggota ke antrian timnya, lalu pastikan timnya masuk ke queue global jika belum ada.
 - b. Dequeue (D): ambil anggota paling depan dari tim paling depan; jika timnya kosong, keluarkan tim dari queue global

.Perbedaan antara kode 1 dan kode 2 adalah:

1. Struktur data

Kode 1	Kode 2
unordered_map<int,int> team = mapping anggota ke tim	map<int,int> on_team = mapping anggota ke tim
unordered_map<int,queue<int>> members = queue per tim	queue<int> each_team[1001] = array queue untuk tiap tim
unordered_map<int,bool> inqueue = cek apakah tim sudah ada di queue global	queue<int> team = queue global
queue<int> torder = queue global (tim)	Kelebihan: lebih efisien (array queue per tim), lookup lebih cepat dan memory lebih hemat
Kelebihan: bisa menangani ID anggota besar, tidak berurutan, bahkan negatif	Kekurangan: butuh batas maksimal jumlah tim (1001). Tidak fleksibel untuk jumlah tim yang besar atau dinamis.
Kekurangan: overhead besar karena unordered_map dipakai untuk semua, termasuk queue tiap tim	

2. Perintah

Tidak banyak perbedaan, hanya posisi nya yang dibalik.

3. Implementasi cek apakah tim sudah dalam antrian global

- a. Kode 1: pakai inqueue[tid] (boolean).
- b. Kode 2: pakai cara implisit, kalau queue tim masih kosong berarti tim tidak ada di antrian global.

Nama : Gilbran Mahdavia Raja
NRP : 5025241134

Kekurangan kode 1 dibanding kode 2:

1. Overhead memori dan performa:
 - a. `unordered_map<int, queue<int>>` lebih berat dibanding `queue<int>` sederhana.
 - b. `inqueue` adalah redundansi (kode 2 tidak perlu, cukup cek kosong/tidak).
2. Lebih rumit dari sisi logika:
 - a. Perlu tracking tambahan `inqueue`.
 - b. Kode 2 lebih ringkas karena logikanya langsung “kalau `queue` tim kosong berarti belum ada di antrian global”.
3. Lebih rawan bug jika tim ada banyak:
 - a. `unordered_map` bisa lambat karena collision hashing, sedangkan `array` `each_team` jauh lebih stabil.
 - b. Untuk problem kompetitif, kode 2 lebih cepat dan aman.